

INFOTACT SOLUTIONS PROJECT DOCUMENT – ADVANCED MERN STACK ENGINEERING

Real-World Applications - Hands-on System Building

Project 1 - "NexusFlow": Visual IoT Telemetry & Rule Engine

Problem Statement: *Hardcoding logic for IoT devices (e.g., "If temperature > 80°C, send an alert") requires developer intervention for every business rule change. Standard MERN apps struggle to store and query high-frequency machine data efficiently while simultaneously running dynamic, user-defined logic against continuous data streams.*

Use Case: A factory floor manager logs into **NexusFlow**. Using a React-based visual drag-and-drop canvas, they connect a "Turbine Sensor" node to a "Moving Average Filter" node, and finally to an "SMS Alert" node. The Node.js backend instantly compiles this visual graph into an executable data stream. As millions of telemetry points pour into the optimized MongoDB Time-Series collections, the custom rule executes in real-time, instantly notifying the manager when anomalies occur.

Key Modules:

- **Visual Graph Builder (React & React Flow):** A node-based UI allowing non-technical users to build complex logic trees and data pipelines.
- **Time-Series Database (MongoDB 5.0+):** Utilizes native MongoDB Time-Series collections optimized for high-ingest, append-only sensor data.
- **Stream Compiler (Node.js & RxJS):** Backend engine that translates the JSON graph from the frontend into highly concurrent, reactive data streams (Observables).
- **Ingestion API (Express & WebSockets):** High-throughput endpoints to receive mock hardware telemetry and broadcast alerts back to the client.

Week-wise Development Plan:

Week	Backend (Node.js, Express, MongoDB Time-Series)	Frontend (React, React Flow)
Week 1	Time-Series Setup: Design MongoDB Time-Series collections for hardware telemetry. Build high-speed Express ingestion endpoints.	Canvas Scaffolding: Initialize React app. Integrate React Flow to build the drag-and-drop node canvas.

Week 2	Logic Compiler: Build the Node.js engine that parses a saved React Flow JSON graph and converts it into executable RxJS data streams.	Node Library: Create custom UI nodes for "Data Sources", "Math Operations", and "Action Triggers".
Mid-Project Review	Ingestion Audit: Demonstrate MongoDB Time-Series collections handling 5,000 writes/sec with minimal storage footprint.	Graph Serialization: Prove the frontend can accurately serialize complex visual graphs into valid JSON for the backend.
Week 3	Live Rule Execution: Connect the incoming WebSocket telemetry to the dynamically compiled RxJS pipelines to evaluate rules in memory.	Live Dashboards: Build real-time Recharts/Chart.js components that subscribe to WebSocket streams for live data visualization.
Week 4	Webhooks & Alerting: Add outbound integration capabilities (e.g., triggering external APIs or mock SMS services when rules evaluate to true).	Refine & Polish: Add visual feedback to the canvas (e.g., making the connection wires "glow" when data passes through them).
Final Project Review	A highly advanced dynamic rule engine capable of executing user-defined logic against high-velocity data streams.	A sophisticated, no-code visual programming interface built entirely in the browser.

Project 2 - "ProxiSpeak": Geospatial Proximity Audio Ecosystem

Problem Statement: Remote collaboration tools (like Zoom) force users into rigid "rooms" where only one person can speak at a time. They lack the spontaneous, spatial dynamics of a real office where you can hear people talking nearby and organically walk over to join a conversation.

Use Case: An Infotact team logs into **ProxiSpeak**. They see a 2D overhead map of a virtual office rendered in React Canvas. As a user moves their avatar closer to a group of colleagues, the WebRTC audio dynamically crossfades, becoming louder. The Node.js backend continuously calculates the spatial distances between all avatars using MongoDB Geospatial queries, orchestrating a seamless, organic audio experience for 50+ concurrent users in the same virtual space.

Key Modules:

- **Geospatial Engine (MongoDB \$near & \$geoWithin):** Tracks the real-time X/Y coordinates of all users and calculates physical proximity clusters.
- **Signaling Server (Node.js & Socket.io):** Orchestrates the WebRTC peer-to-peer connection handshakes based on who is within "earshot."

- **Spatial Audio Mixer (Web Audio API):** Frontend module that dynamically adjusts the gain (volume) and 3D panning (left/right ear) of peer audio tracks based on relative avatar distance.
- **2D Virtual World (React & HTML5 Canvas):** A high-performance rendering engine for the virtual office map and real-time avatar movements.

Week-wise Development Plan:

Week	Backend (Node.js, Express, Socket.io, MongoDB)	Frontend (React, WebRTC, Canvas API)
Week 1	Signaling & Movement: Build WebSocket server to broadcast avatar X/Y coordinates. Setup basic MongoDB geospatial schemas.	World Rendering: Build the React Canvas layer. Implement smooth keyboard/mouse avatar movement and coordinate broadcasting.
Week 2	Proximity Calculations: Implement MongoDB \$near queries to group users. Logic to only send WebRTC offer/answers to users within a 100-pixel radius.	WebRTC Integration: Implement standard WebRTC peer-to-peer audio connections using the browser's getUserMedia.
Mid-Project Review	Spatial Audit: Prove the backend accurately filters and clusters users based on geospatial coordinates under 20ms.	Audio Check: Demonstrate two browsers successfully establishing a WebRTC audio connection via the signaling server.
Week 3	Mesh Scaling: Optimize the WebRTC mesh network. Implement Redis Pub/Sub to allow scaling the Socket.io server across multiple Node instances.	Dynamic Attenuation: Hook WebRTC audio tracks into the Web Audio API. Write algorithms to calculate volume drop-off based on pixel distance.
Week 4	Room Persistence: Save office layouts and furniture obstacles (which block sound) into MongoDB.	Refine & Polish: Add 3D audio panning (stereo separation). Ensure Canvas animations render at a steady 60 FPS.
Final Project Review	A highly scalable, stateful geospatial server managing complex P2P routing logic.	An immersive, physics-aware communication platform demonstrating advanced browser audio manipulation.

Project 3 - "StreamWeaver": High-Throughput No-Code ETL Pipeline

Problem Statement: *Uploading and transforming massive datasets (e.g., 5GB CSV or JSON files) typically crashes web browsers or exhausts the Node.js V8 memory limit (Heap Out of Memory). Non-technical users need a way to upload, clean, and map massive datasets into databases without writing scripts.*

Use Case: A data analyst uploads a 5-million-row CSV into **StreamWeaver**. Instead of loading the file into RAM, the React frontend streams the file to Node.js in chunks. The user utilizes a visual UI to map "Column A" to "First Name" and applies a custom JavaScript transformation to capitalize it. Node.js processes the data utilizing native Streams (stream.Transform), executes the user's custom mapping securely in an isolated V8 sandbox, and bulk-inserts the cleaned data into MongoDB—all while the frontend displays a live progress bar without freezing.

Key Modules:

- **Native Node Streams (fs & stream):** Bypasses memory limits by utilizing piping and transform streams to process gigabytes of data piece-by-piece.
- **Secure Sandbox (isolated-vm):** Executes user-defined JavaScript transformation rules securely on the backend without exposing the Node environment to malicious code.
- **Bulk Operations (MongoDB bulkWrite):** Caches processed data chunks and inserts them in batches of 1,000 to maximize database write throughput.
- **Virtualization (React Virtualized):** Renders a preview grid of massive datasets in the DOM without crashing the browser by only rendering the rows currently visible on screen.

Week-wise Development Plan:

Week	Backend (Node.js, Express Streams, MongoDB)	Frontend (React, React Virtualized)
Week 1	Multipart Streaming: Implement busboy or multer to accept massive file uploads via streams (never storing the full file in memory).	Virtual Grid: Implement react-window or react-virtualized to display a preview of the first 1,000 rows of a CSV.
Week 2	ETL Transform Streams: Write Node.js stream.Transform classes to parse CSV chunks on the fly and convert them to JSON objects.	Mapping UI: Build the interface allowing users to map source columns to destination MongoDB fields visually.
Mid-Project Review	Memory Audit: Upload a 2GB file and prove via Node.js memory profiling that the server RAM never exceeds 150MB.	UI Performance: Ensure the virtualized data grid scrolls smoothly without DOM lag.
Week 3	Sandboxed Execution: Integrate isolated-vm to allow users to write inline JavaScript (e.g., return value.toUpperCase()) and apply it safely during the stream.	Live Progress: Connect a WebSocket to stream processing metadata back to the client, displaying a live progress bar and "rows processed/sec" metric.

Week 4	Database Bulk Ingestion: Implement a buffer mechanism that triggers MongoDB insertMany or bulkWrite operations every 5,000 records.	Refine & Polish: Build an error-handling UI layer to highlight rows that failed validation during the streaming process.
Final Project Review	An enterprise-grade, memory-safe ETL architecture utilizing deep Node.js core modules.	A highly robust UI handling massive data interactions flawlessly.

Common Features Across All Projects

- **Beyond Standard REST (Streams & Sockets):** Moving past basic request/response cycles. These projects heavily utilize Node.js native streams, WebRTC, and WebSockets to handle continuous, high-throughput, and bi-directional data flow.
- **Advanced MongoDB Operations:** Utilizing advanced database features natively within the MERN stack, including Time-Series collections, \$geoNear Geospatial indexing, and high-performance bulkWrite buffering.
- **Heavy Frontend Processing:** Shifting complex workloads to the browser safely, utilizing HTML5 Canvas APIs, Web Audio APIs for spatial mixing, and DOM Virtualization for massive datasets.
- **Strict Performance Profiling:** Architecture designed specifically to avoid V8 engine memory exhaustion and prevent browser main-thread blocking (ensuring 60 FPS rendering).

Overall Project Summary

Project	Target Business Domain	Technology Focus
"NexusFlow": Visual Rule Engine	IoT & Automation	React Flow, Node.js RxJS, MongoDB Time-Series, WebSockets
"ProxiSpeak": Spatial Audio	Collaboration & Metaverse	WebRTC, Web Audio API, HTML5 Canvas, MongoDB Geospatial
"StreamWeaver": No-Code ETL	Data Engineering & Ops	Node.js Streams, isolated-vm, MongoDB Bulk Ops, React Virtualization